

Chương 1 – Bức tranh tổng quan về Machine Learning

Notebook này chứa các ví dụ mã nguồn trong chương 1. Bạn cũng sẽ tìm thấy lời giải cho các bài tập ở cuối notebook. Phần còn lại của notebook này được sử dụng để tạo tệp `lifesat.csv` từ các nguồn dữ liệu gốc và một số hình vẽ của chương này.

Bạn có thể xem qua mã nguồn trong notebook này nếu muốn, nhưng nội dung thực sự sẽ bắt đầu từ chương sau.

 [Open in Colab](#)  [Open in Kaggle](#)

Thiết lập

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Hãy xác định kích thước phông chữ mặc định để vẽ các biểu đồ đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=12)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=12)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Đảm bảo kết quả của notebook ổn định qua các lần chạy:

```
import numpy as np

np.random.seed(42)
```

Ví dụ mã nguồn 1-1

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Download and prepare the data
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

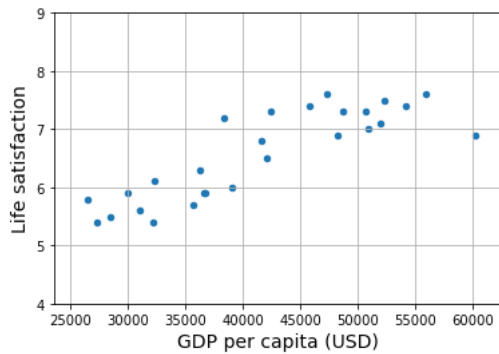
# Visualize the data
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Select a linear model
model = LinearRegression()

# Train the model
```

```
model.fit(X, y)
```

```
# Make a prediction for Cyprus
X_new = [[37_655.2]] # Cyprus' GDP per capita in 2020
print(model.predict(X_new)) # outputs [[6.30165767]]
```



```
[[6.30165767]]
```

Việc thay thế mô hình Hồi quy Tuyến tính (Linear Regression) bằng mô hình Hồi quy k-Hàng xóm gần nhất (k-Nearest Neighbors regression - trong ví dụ này k = 3) trong đoạn mã trước đơn giản chỉ là thay thế hai dòng này:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
```

với hai dòng sau:

```
from sklearn.neighbors import KNeighborsRegressor

model = KNeighborsRegressor(n_neighbors=3)
```

```
# Select a 3-Nearest Neighbors regression model
from sklearn.neighbors import KNeighborsRegressor

model = KNeighborsRegressor(n_neighbors=3)

# Train the model
model.fit(X, y)

# Make a prediction for Cyprus
print(model.predict(X_new)) # outputs [[6.33333333]]
```

```
[[6.33333333]]
```

▼ Tạo dữ liệu và hình ảnh — có thể bỏ qua

Đây là đoạn mã tôi đã sử dụng để tạo tập dữ liệu `lifesat.csv`. Bạn có thể an tâm bỏ qua phần này.

Tạo một hàm để lưu các hình ảnh:

```
from pathlib import Path

# Where to save the figures
IMAGES_PATH = Path() / "images" / "fundamentals"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

▼ Tải và chuẩn bị dữ liệu sự hài lòng cuộc sống (Life satisfaction)

Để tạo `lifesat.csv`, tôi đã tải dữ liệu Better Life Index (BLI) từ [website của OECD](#) (để lấy chỉ số Hạnh phúc cuộc sống của mỗi quốc gia), và dữ liệu GDP đầu người từ Ngân hàng Thế giới tại [OurWorldInData.org](#). Dữ liệu BLI nằm trong `datasets/lifesat/oecd_bli.csv` (dữ liệu từ năm 2020), và dữ liệu GDP đầu người nằm trong `datasets/lifesat/gdp_per_capita.csv` (dữ liệu đến năm 2020).

Nếu bạn muốn lấy các phiên bản mới nhất, hãy thoải mái thực hiện. Tuy nhiên, có thể có một số thay đổi (ví dụ: tên cột, hoặc các quốc gia khác nhau bị thiếu dữ liệu), vì vậy hãy chuẩn bị tinh thần để điều chỉnh mã nguồn.

```
import urllib.request

datapath = Path() / "datasets" / "lifesat"
datapath.mkdir(parents=True, exist_ok=True)

data_root = "https://github.com/ageron/data/raw/main/"
for filename in ("oecd_bli.csv", "gdp_per_capita.csv"):
    if not (datapath / filename).is_file():
        print("Downloading", filename)
        url = data_root + "lifesat/" + filename
        urllib.request.urlretrieve(url, datapath / filename)
```

```
Downloading oecd_bli.csv
Downloading gdp_per_capita.csv
```

```
oecd_bli = pd.read_csv(datapath / "oecd_bli.csv")
gdp_per_capita = pd.read_csv(datapath / "gdp_per_capita.csv")
```

Tiền xử lý dữ liệu GDP đầu người để chỉ giữ lại năm 2020:

```
gdp_year = 2020
gdppc_col = "GDP per capita (USD)"
lifesat_col = "Life satisfaction"

gdp_per_capita = gdp_per_capita[gdp_per_capita["Year"] == gdp_year]
gdp_per_capita = gdp_per_capita.drop(["Code", "Year"], axis=1)
gdp_per_capita.columns = ["Country", gdppc_col]
gdp_per_capita.set_index("Country", inplace=True)

gdp_per_capita.head()
```

	GDP per capita (USD)
Country	
Afghanistan	1978.961579
Africa Eastern and Southern	3387.594670
Africa Western and Central	4003.158913
Albania	13295.410885
Algeria	10681.679297

Tiền xử lý dữ liệu OECD BLI để chỉ giữ lại cột `Life satisfaction`:

```
oecd_bli = oecd_bli[oecd_bli["INEQUALITY"]=="TOT"]
oecd_bli = oecd_bli.pivot(index="Country", columns="Indicator", values="Value")

oecd_bli.head()
```

Indicator	Air pollution	Dwellings without basic facilities	Educational attainment	Employees working very long hours	Employment rate	Feeling safe walking alone at night	Homicide rate	Household net adjusted disposable income	Household net wealth	Housing expenditure	...
Country											
Australia	5.0	NaN	81.0	13.04	73.0	63.5	1.1	32759.0	427064.0	20.0	...
Austria	16.0	0.9	85.0	6.66	72.0	80.6	0.5	33541.0	308325.0	21.0	...
Belgium	15.0	1.9	77.0	4.75	63.0	70.1	1.0	30364.0	386006.0	21.0	...
Brazil	10.0	6.7	49.0	7.13	61.0	35.6	26.7	NaN	NaN	NaN	...
Canada	7.0	0.2	91.0	3.69	73.0	82.2	1.3	30854.0	423849.0	22.0	...

5 rows × 24 columns

Bây giờ hãy gộp dữ liệu hài lòng cuộc sống và dữ liệu GDP đầu người, chỉ giữ lại các cột GDP per capita và Life satisfaction:

```

full_country_stats = pd.merge(left=oced_bli, right=gdp_per_capita,
                              left_index=True, right_index=True)
full_country_stats.sort_values(by=gdppc_col, inplace=True)
full_country_stats = full_country_stats[[gdppc_col, lifesat_col]]

full_country_stats.head()

```

	GDP per capita (USD)	Life satisfaction
Country		
South Africa	11466.189672	4.7
Colombia	13441.492952	6.3
Brazil	14063.982505	6.4
Mexico	17887.750736	6.5
Chile	23324.524751	6.5

Để minh họa rủi ro của việc quá khớp (overfitting), tôi chỉ sử dụng một phần dữ liệu trong hầu hết các hình vẽ (tất cả các quốc gia có GDP đầu người nằm trong khoảng `min_gdp` và `max_gdp`). Sau đó trong chương này, tôi sẽ tiết lộ các quốc gia bị thiếu và chỉ ra rằng chúng không tuân theo xu hướng tuyến tính tương tự.

```

min_gdp = 23_500
max_gdp = 62_500

country_stats = full_country_stats[(full_country_stats[gdppc_col] >= min_gdp) &
                                   (full_country_stats[gdppc_col] <= max_gdp)]

country_stats.head()

```

	GDP per capita (USD)	Life satisfaction
Country		
Russia	26456.387938	5.8
Greece	27287.083401	5.4
Turkey	28384.987785	5.5
Latvia	29932.493910	5.9
Hungary	31007.768407	5.6

```

country_stats.to_csv(datapath / "lifesat.csv")
full_country_stats.to_csv(datapath / "lifesat_full.csv")

```

```

country_stats.plot(kind='scatter', figsize=(5, 3), grid=True,
                  x=gdppc_col, y=lifesat_col)

```

```

min_life_sat = 4
max_life_sat = 9

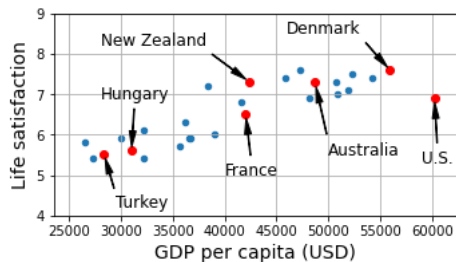
position_text = {
    "Turkey": (29_500, 4.2),
    "Hungary": (28_000, 6.9),
    "France": (40_000, 5),
    "New Zealand": (28_000, 8.2),
    "Australia": (50_000, 5.5),
    "United States": (59_000, 5.3),
    "Denmark": (46_000, 8.5)
}

for country, pos_text in position_text.items():
    pos_data_x = country_stats[gdppc_col].loc[country]
    pos_data_y = country_stats[lifesat_col].loc[country]
    country = "U.S." if country == "United States" else country
    plt.annotate(country, xy=(pos_data_x, pos_data_y),
                 xytext=pos_text, fontsize=12,
                 arrowprops=dict(facecolor='black', width=0.5,
                                 shrink=0.08, headwidth=5))
    plt.plot(pos_data_x, pos_data_y, "ro")

plt.axis([min_gdp, max_gdp, min_life_sat, max_life_sat])

save_fig('money_happy_scatterplot')
plt.show()

```



```

highlighted_countries = country_stats.loc[list(position_text.keys())]
highlighted_countries[[gdppc_col, lifesat_col]].sort_values(by=gdppc_col)

```

	GDP per capita (USD)	Life satisfaction
Country		
Turkey	28384.987785	5.5
Hungary	31007.768407	5.6
France	42025.617373	6.5
New Zealand	42404.393738	7.3
Australia	48697.837028	7.3
Denmark	55938.212809	7.6
United States	60235.728492	6.9

```

country_stats.plot(kind='scatter', figsize=(5, 3), grid=True,
                  x=gdppc_col, y=lifesat_col)

X = np.linspace(min_gdp, max_gdp, 1000)

w1, w2 = 4.2, 0
plt.plot(X, w1 + w2 * 1e-5 * X, "r")
plt.text(40_000, 4.9, fr"$\theta_0 = {w1}$", color="r")
plt.text(40_000, 4.4, fr"$\theta_1 = {w2}$", color="r")

w1, w2 = 10, -9
plt.plot(X, w1 + w2 * 1e-5 * X, "g")
plt.text(26_000, 8.5, fr"$\theta_0 = {w1}$", color="g")
plt.text(26_000, 8.0, fr"$\theta_1 = {w2} \times 10^{{-5}}$", color="g")

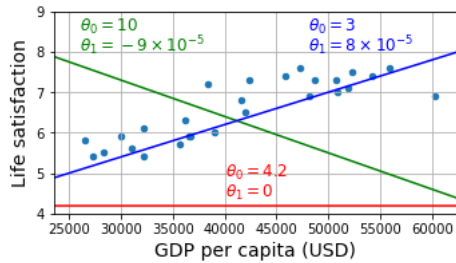
w1, w2 = 3, 8
plt.plot(X, w1 + w2 * 1e-5 * X, "b")

```

```
plt.text(48_000, 8.5, fr"\theta_0 = {w1}$", color="b")
plt.text(48_000, 8.0, fr"\theta_1 = {w2} \times 10^{{-5}}$", color="b")

plt.axis([min_gdp, max_gdp, min_life_sat, max_life_sat])

save_fig('tweaking_model_params_plot')
plt.show()
```



```
from sklearn import linear_model

X_sample = country_stats[[gdppc_col]].values
y_sample = country_stats[[lifesat_col]].values

lin1 = linear_model.LinearRegression()
lin1.fit(X_sample, y_sample)

t0, t1 = lin1.intercept_[0], lin1.coef_.ravel()[0]
print(f"\theta_0={t0:.2f}, \theta_1={t1:.2e}")

\theta_0=3.75, \theta_1=6.78e-05
```

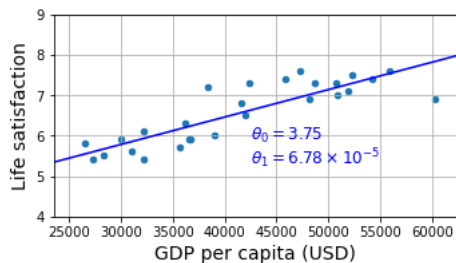
```
country_stats.plot(kind='scatter', figsize=(5, 3), grid=True,
                  x=gdppc_col, y=lifesat_col)

X = np.linspace(min_gdp, max_gdp, 1000)
plt.plot(X, t0 + t1 * X, "b")

plt.text(max_gdp - 20_000, min_life_sat + 1.9,
        fr"\theta_0 = {t0:.2f}$", color="b")
plt.text(max_gdp - 20_000, min_life_sat + 1.3,
        fr"\theta_1 = {t1 * 1e5:.2f} \times 10^{{-5}}$", color="b")

plt.axis([min_gdp, max_gdp, min_life_sat, max_life_sat])

save_fig('best_fit_model_plot')
plt.show()
```



```
cyprus_gdp_per_capita = gdp_per_capita[gdppc_col].loc["Cyprus"]
cyprus_gdp_per_capita
```

```
37655.1803457421
```

```
cyprus_predicted_life_satisfaction = lin1.predict([[cyprus_gdp_per_capita]])[0, 0]
cyprus_predicted_life_satisfaction
```

```
6.301656332738056
```

```
country_stats.plot(kind='scatter', figsize=(5, 3), grid=True,
                  x=gdppc_col, y=lifesat_col)

X = np.linspace(min_gdp, max_gdp, 1000)
```

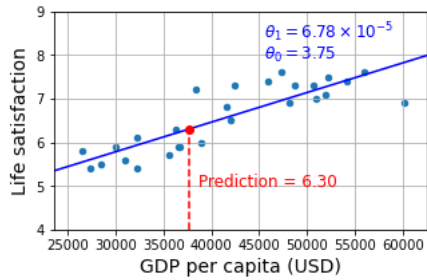
```
plt.plot(X, t0 + t1 * X, "b")

plt.text(min_gdp + 22_000, max_life_sat - 1.1,
         fr"$\theta_0 = {t0:.2f}$", color="b")
plt.text(min_gdp + 22_000, max_life_sat - 0.6,
         fr"$\theta_1 = {t1 * 1e5:.2f} \times 10^{{-5}}$", color="b")

plt.plot([cyprus_gdp_per_capita, cyprus_gdp_per_capita],
         [min_life_sat, cyprus_predicted_life_satisfaction], "r--")
plt.text(cyprus_gdp_per_capita + 1000, 5.0,
         fr"Prediction = {cyprus_predicted_life_satisfaction:.2f}", color="r")
plt.plot(cyprus_gdp_per_capita, cyprus_predicted_life_satisfaction, "ro")

plt.axis([min_gdp, max_gdp, min_life_sat, max_life_sat])

plt.show()
```



```
missing_data = full_country_stats[(full_country_stats[gdpcc_col] < min_gdp) |
                                   (full_country_stats[gdpcc_col] > max_gdp)]

missing_data
```

	GDP per capita (USD)	Life satisfaction
Country		
South Africa	11466.189672	4.7
Colombia	13441.492952	6.3
Brazil	14063.982505	6.4
Mexico	17887.750736	6.5
Chile	23324.524751	6.5
Norway	63585.903514	7.6
Switzerland	68393.306004	7.5
Ireland	89688.956958	7.0
Luxembourg	110261.157353	6.9

```
position_text_missing_countries = {
    "South Africa": (20_000, 4.2),
    "Colombia": (6_000, 8.2),
    "Brazil": (18_000, 7.8),
    "Mexico": (24_000, 7.4),
    "Chile": (30_000, 7.0),
    "Norway": (51_000, 6.2),
    "Switzerland": (62_000, 5.7),
    "Ireland": (81_000, 5.2),
    "Luxembourg": (92_000, 4.7),
}
```

```
full_country_stats.plot(kind='scatter', figsize=(8, 3),
                        x=gdpcc_col, y=lifesat_col, grid=True)

for country, pos_text in position_text_missing_countries.items():
    pos_data_x, pos_data_y = missing_data.loc[country]
    plt.annotate(country, xy=(pos_data_x, pos_data_y),
                 xytext=pos_text, fontsize=12,
                 arrowprops=dict(facecolor='black', width=0.5,
                                 shrink=0.08, headwidth=5))
plt.plot(pos_data_x, pos_data_y, "rs")
```

```

X = np.linspace(0, 115_000, 1000)
plt.plot(X, t0 + t1 * X, "b:")

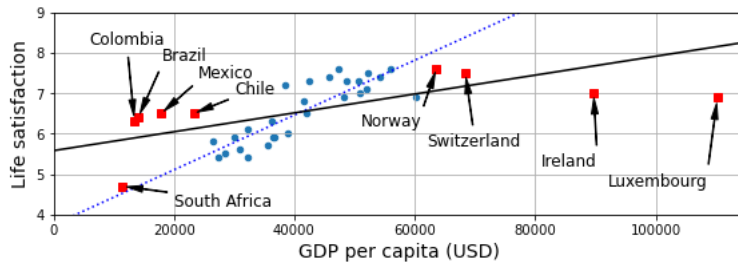
lin_reg_full = linear_model.LinearRegression()
Xfull = np.c_[full_country_stats[gdppc_col]]
yfull = np.c_[full_country_stats[lifesat_col]]
lin_reg_full.fit(Xfull, yfull)

t0full, t1full = lin_reg_full.intercept_[0], lin_reg_full.coef_.ravel()[0]
X = np.linspace(0, 115_000, 1000)
plt.plot(X, t0full + t1full * X, "k")

plt.axis([0, 115_000, min_life_sat, max_life_sat])

save_fig('representative_training_data_scatterplot')
plt.show()

```



```

from sklearn import preprocessing
from sklearn import pipeline

full_country_stats.plot(kind='scatter', figsize=(8, 3),
                        x=gdppc_col, y=lifesat_col, grid=True)

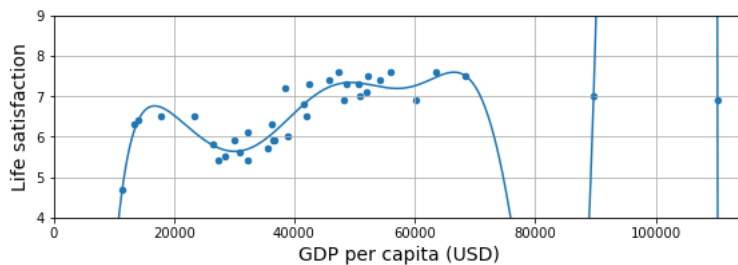
poly = preprocessing.PolynomialFeatures(degree=10, include_bias=False)
scaler = preprocessing.StandardScaler()
lin_reg2 = linear_model.LinearRegression()

pipeline_reg = pipeline.Pipeline([
    ('poly', poly),
    ('scal', scaler),
    ('lin', lin_reg2)])
pipeline_reg.fit(Xfull, yfull)
curve = pipeline_reg.predict(X[:, np.newaxis])
plt.plot(X, curve)

plt.axis([0, 115_000, min_life_sat, max_life_sat])

save_fig('overfitting_model_plot')
plt.show()

```



```

w_countries = [c for c in full_country_stats.index if "W" in c.upper()]
full_country_stats.loc[w_countries][lifesat_col]

```

```

Country
New Zealand    7.3
Sweden         7.3
Norway         7.6
Switzerland    7.5
Name: Life satisfaction, dtype: float64

```



```
all_w_countries = [c for c in gdp_per_capita.index if "W" in c.upper()]
gdp_per_capita.loc[all_w_countries].sort_values(by=gdppc_col)
```

GDP per capita (USD)	
Country	
Malawi	1486.778248
Rwanda	2098.710362
Zimbabwe	2744.690758
Africa Western and Central	4003.158913
Papua New Guinea	4101.218882
Lower middle income	6722.809932
Eswatini	8392.717564
Low & middle income	10293.855325
Arab World	13753.707307
Botswana	16040.008473
World	16194.040310
New Zealand	42404.393738
Sweden	50683.323510
Norway	63585.903514
Switzerland	68393.306004

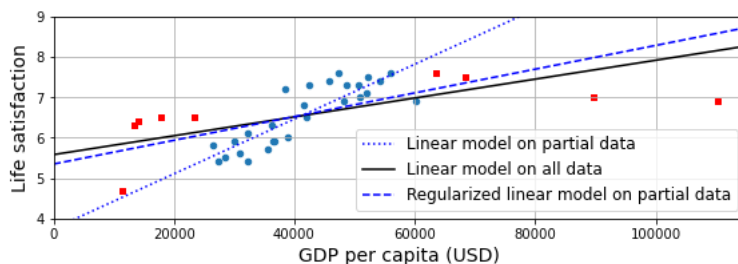
```
country_stats.plot(kind='scatter', x=gdppc_col, y=lifesat_col, figsize=(8, 3))
missing_data.plot(kind='scatter', x=gdppc_col, y=lifesat_col,
                             marker="s", color="r", grid=True, ax=plt.gca())
```

```
X = np.linspace(0, 115_000, 1000)
plt.plot(X, t0 + t1*X, "b:", label="Linear model on partial data")
plt.plot(X, t0full + t1full * X, "k-", label="Linear model on all data")
```

```
ridge = linear_model.Ridge(alpha=10**9.5)
X_sample = country_stats[[gdppc_col]]
y_sample = country_stats[[lifesat_col]]
ridge.fit(X_sample, y_sample)
t0ridge, t1ridge = ridge.intercept_[0], ridge.coef_.ravel()[0]
plt.plot(X, t0ridge + t1ridge * X, "b--",
         label="Regularized linear model on partial data")
plt.legend(loc="lower right")
```

```
plt.axis([0, 115_000, min_life_sat, max_life_sat])
```

```
save_fig('ridge_model_plot')
plt.show()
```



✧ Lời giải bài tập

1. Machine Learning là việc xây dựng các hệ thống có thể học hỏi từ dữ liệu. Học có nghĩa là trở nên tốt hơn ở một nhiệm vụ nào đó, dựa trên một thước đo hiệu suất.
2. Machine Learning tuyệt vời cho các vấn đề phức tạp mà chúng ta không có giải pháp thuật toán, để thay thế danh sách dài các quy tắc được tinh chỉnh thủ công, để xây dựng các hệ thống thích ứng với môi trường biến động và cuối cùng là giúp con người học hỏi

(ví dụ: khai phá dữ liệu).

3. Một tập huấn luyện có nhãn (labeled training set) là tập huấn luyện chứa giải pháp mong muốn (còn gọi là nhãn) cho mỗi trường hợp.
4. Hai nhiệm vụ có giám sát phổ biến nhất là hồi quy (regression) và phân loại (classification).
5. Các nhiệm vụ không giám sát phổ biến bao gồm phân cụm (clustering), trực quan hóa, giảm chiều dữ liệu và học luật kết hợp.
6. Học tăng cường (Reinforcement Learning) có khả năng hoạt động tốt nhất nếu chúng ta muốn một robot học cách đi bộ trên nhiều địa hình không xác định.
7. Nếu bạn không biết cách xác định các nhóm, bạn có thể sử dụng thuật toán phân cụm (học không giám sát). Nếu bạn biết các nhóm bạn muốn có, bạn có thể sử dụng thuật toán phân loại (học có giám sát).
8. Phát hiện thư rác (Spam detection) là một bài toán học có giám sát điển hình.
9. Hệ thống học trực tuyến (online learning) có thể học dần dần (incrementally), trái ngược với hệ thống học theo lô (batch learning).
10. Các thuật toán ngoài bộ nhớ (Out-of-core) có thể xử lý lượng dữ liệu khổng lồ không thể vừa với bộ nhớ chính của máy tính.
11. Hệ thống học dựa trên phiên bản (instance-based) ghi nhớ dữ liệu huấn luyện; sau đó sử dụng thước đo độ tương đồng để đưa ra dự đoán.
12. Một mô hình có một hoặc nhiều tham số mô hình xác định những gì nó sẽ dự đoán. Một siêu tham số (hyperparameter) là một tham số của chính thuật toán học, không phải của mô hình.
13. Các thuật toán học dựa trên mô hình tìm kiếm giá trị tối ưu cho các tham số mô hình để mô hình khái quát hóa tốt. Chúng ta thường huấn luyện bằng cách giảm thiểu một hàm chi phí.
14. Một số thách thức chính trong Machine Learning là thiếu dữ liệu, chất lượng dữ liệu kém, dữ liệu không mang tính đại diện, các đặc trưng không mang tính thông tin, mô hình quá đơn giản gây ra hiện tượng dưới khớp (underfitting) và mô hình quá phức tạp gây ra hiện tượng quá khớp (overfitting).
15. Nếu mô hình hoạt động tốt trên dữ liệu huấn luyện nhưng khái quát hóa kém trên các trường hợp mới, mô hình đó có khả năng bị quá khớp. Giải pháp là lấy thêm dữ liệu, đơn giản hóa mô hình hoặc giảm nhiễu.
16. Tập kiểm tra (test set) được sử dụng để ước tính lỗi khái quát hóa mà mô hình sẽ mắc phải trên các trường hợp mới.
17. Tập xác thực (validation set) được sử dụng để so sánh các mô hình, giúp chọn mô hình tốt nhất và tinh chỉnh các siêu tham số.
18. Tập train-dev được sử dụng khi có nguy cơ không khớp giữa dữ liệu huấn luyện và dữ liệu dùng trong tập xác thực/kiểm tra.
19. Nếu bạn tinh chỉnh siêu tham số bằng tập kiểm tra, bạn có nguy cơ quá khớp với tập kiểm tra và lỗi khái quát hóa đo được sẽ lạc quan quá mức.

Bắt đầu lập trình hoặc tạo mã bằng trí tuệ nhân tạo (AI).